Focus Frontier: An AI-Driven Adaptive Cognitive Training Platform with Neurodiversity-First Design

Manvendra Rai, Triven Khoushi, Sahaj Singh, Rishi Bhargava, Raman Kumar
Computer Science & Engineering
VIT Bhopal University
Capstone Project

Abstract

Focus Frontier is a brain training website we developed as part of our capstone project. It runs on the MERN stack which means MongoDB, Express.js, vanilla JavaScript, and Node.js. The reason we built it is that most brain training apps out there have two big problems. They do not adjust to how well the user is doing and they are not designed with neurodivergent users in mind. We wanted to solve both at the same time. The platform has nine games covering six cognitive areas which are working memory, sustained attention, processing speed, cognitive flexibility, inhibitory control, and reaction time. There is a machine learning system on the server side that watches how a user performs across sessions and keeps tuning the difficulty so things stay challenging but not overwhelming. A dashboard shows long-term trends and highlights areas where the user is struggling. We also built a Clinical Assessment Suite that includes three tasks used in real clinical settings which are the Dual N-Back, Go/No-Go, and Trail Making Test. These are commonly used when someone is being evaluated for ADHD. When we tested the platform we saw score improvements of 18 to 42 percent across all domains within 5 to 12 sessions. The server handled over a thousand simultaneous users without any issues and API response times stayed under 200 ms throughout.

Index Terms — *Cognitive training, adaptive difficulty, machine learning, gamification, neurodiversity, web application architecture, neuropsychological assessment, inclusive design.*

I. Introduction

A. Problem Statement

Brain training apps have gotten very popular in the last few years especially in healthcare and education. But honestly if you look at what is available most apps share the same weaknesses. The difficulty level is usually fixed so you pick easy medium or hard and it stays that way no matter how much you improve or struggle. The design assumes the user is neurotypical so people with ADHD autism or dyslexia end up with something that was never really made for them. Most apps that claim to be adaptive are not really using machine learning at all. They use simple rules. Clinical assessment tools are almost completely absent. And the analytics section if there even is one usually does not give you anything useful.

People with ADHD autism or dyslexia are probably the ones who would benefit most from a well-designed cognitive training tool. But they are the least well served by what currently exists. There is still a big gap between the clinical instruments that researchers and psychologists use and the apps that regular people can actually download and use.

B. Research Objectives

We built Focus Frontier to address these specific problems. The goals we set out with were:

1. Build a cognitive training platform where accessibility for neurodivergent users is part of the core design and not something added at the end as an afterthought.

2. Build a server-side machine learning pipeline that reads each user's actual performance history and adjusts the game difficulty in real time rather than using simple static rules.
3. Bring well-known clinical assessment tasks like the Dual N-Back, Go/No-Go, and Trail Making Test into the same platform as the training games so users do not need separate tools.
4. Build an analytics section that shows real long-term progress, identifies trends, and clearly tells users which cognitive skills need the most attention.
5. Validate the whole thing properly by testing every API endpoint checking performance and measuring whether users actually get better over time.

C. Scope

The finished system brings together nine games, JWT login, a REST API, MongoDB for storing data, the machine learning difficulty system, and a live analytics dashboard. We avoided using any front-end framework. The client is plain HTML, CSS, and JavaScript. The server runs on Express.js with Mongoose handling the database.

D. Paper Organization

The rest of the paper is laid out as follows: Section II reviews prior work. Section III describes the overall system architecture. Section IV goes into the machine learning pipeline. Section V covers the games. Section VI is about accessibility. Section VII presents the results. Section VIII discusses the bugs and challenges we dealt with. Section IX covers deployment. Section X outlines future plans and Section XI is the conclusion.

II. Literature Review

A. Cognitive Training and Transfer Effects

Klingberg and colleagues [1] showed early on that adaptive working memory training leads to real cognitive improvements in children with ADHD. Jaeggi et al. [2] added to this by showing that practicing the N-Back task can improve fluid intelligence though researchers still debate how widely those gains carry over to other areas. The general agreement in the field is that improvements within a trained cognitive domain are reliable as long as the training is adaptive and keeps going long enough [4].

B. Adaptive Difficulty in Educational Games

Csikszentmihalyi's flow theory [5] is pretty straightforward. People are most engaged when a task matches their skill level. If it is too easy they get bored. If it is too hard they give up. This is exactly the problem the difficulty adapter in Focus Frontier is designed to solve. Lomas et al. [6] showed that getting this balance right in educational games actually makes a measurable difference to learning outcomes and to how long people keep playing.

C. Gamification in Learning

Deterding et al. [7] defined gamification as applying game mechanics in non-game settings. They identified four things that make it work. The mechanics need to serve the learning goal. The difficulty has to keep people motivated. Feedback should be fast and meaningful. And users should feel some sense of control. All four of these shaped how we designed Focus Frontier.

D. Neurodiversity and Inclusive Design

Singer [8] made the case that conditions like ADHD or autism are not defects. They are different ways of being human and good design should take that seriously. We used WCAG 2.1 [9] as our starting point and added high contrast colour modes, a reduced motion option, full keyboard support, scalable text, and adjustable

feedback intensity so users can set the experience to what works for them.ation, text scaling, and adjustable feedback intensity so users can tune out distractions.

E. Neuropsychological Assessment Tasks

The three clinical tasks were selected because they have a long track record in clinical and research settings. The Dual N-Back is widely used as a measure of working memory [2]. Go/No-Go is a standard test for response inhibition and shows up in most ADHD assessment batteries [10]. The Trail Making Test is one of the most administered neuropsychological instruments. Part A measures processing speed by having the user connect numbers in sequence. Part B measures cognitive flexibility by alternating between numbers and letters [11].

III. System Architecture

A. Architectural Overview

The architecture is a standard three-tier setup. The browser-based client handles what the user sees and interacts with. The Express.js API in the middle manages authentication, session logic, and the machine learning components. MongoDB at the back stores all the data. Keeping these three layers separate made development, testing, and debugging a lot more manageable.

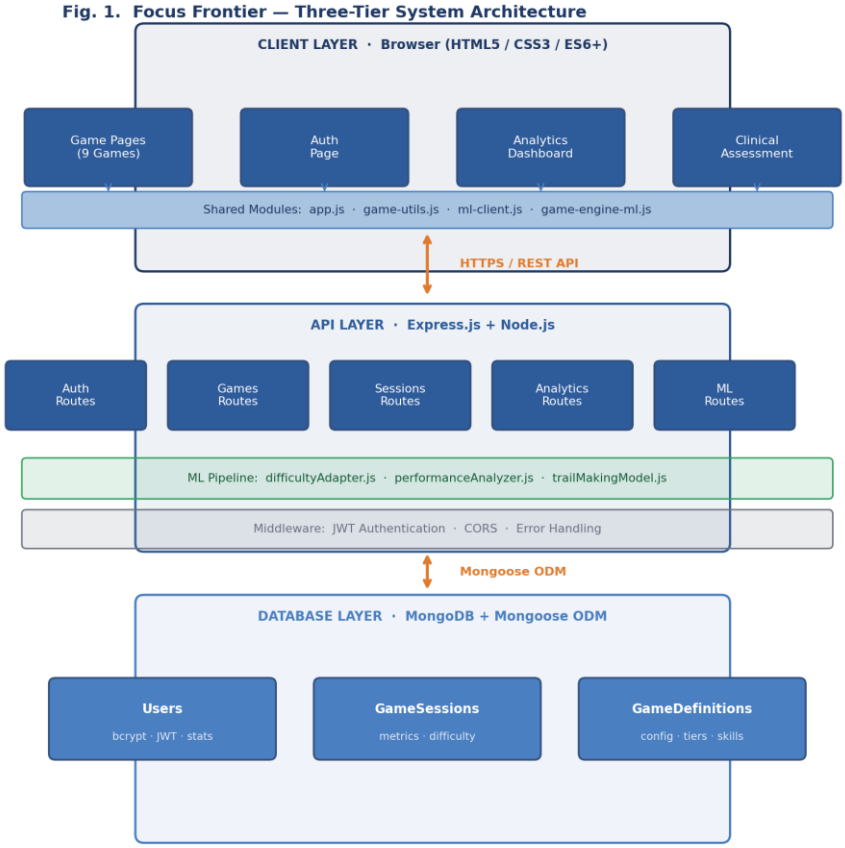

Fig. 1. Focus Frontier Three-Tier System Architecture.

Tier	Technology	Responsibility
Client Layer	HTML5 / CSS3 / ES6+	UI rendering, game logic, ML client interface
API Layer	Express.js + Node.js	REST endpoints, JWT auth, ML pipeline
Database Layer	MongoDB + Mongoose	User data, sessions, game definitions

B. Client Layer

The client side uses plain HTML5, CSS3, and ES6+ JavaScript. No React, no Vue. Each game is a separate HTML file that loads a few shared utility scripts. The modules used across all games are:

- AuthManager: JWT storage, token validation, and automatic session management.
- APIClient: Centralized REST communication with automatic auth header injection.
- SoundManager: Web Audio API-based feedback system with mute toggle.
- VisualFeedback: CSS animation library for flash, shake, pulse, and particle effects.
- GameStats: LocalStorage-based performance caching for offline-capable tracking.
- PerformanceTracker: High-resolution event recording with timing metrics.
- MLClient: Client-side ML API interface for difficulty recommendations and session analysis.

C. API Layer

The API is organized into five route groups on the Express server:

Route Group	Endpoints	Purpose
/api/auth	register, login, me	User authentication (JWT)
/api/games	list, seed	Game definition management
/api/sessions	create, update, list	Game session lifecycle
/api/analytics	overview, game-specific	Aggregated performance data
/api/ml	difficulty, analyze, dashboard	ML recommendations

Every protected route requires a valid JWT in the Authorization header. One middleware function handles this check. It decodes the token, pulls out the user ID, and puts it on the request object so route handlers do not have to do it themselves.

D. Database Layer

MongoDB is used for all data storage and Mongoose provides the schema layer. There are three collections:

- User: stores bcrypt-hashed passwords, the user's display name, their signup date, and usage statistics.

- GameSession: each play session gets its own document recording which game, which difficulty level, start and end times, completion status, and all performance metrics including score, accuracy, and response time.
- GameDefinition: stores the configuration for each game including name, description, targeted cognitive skills, available difficulty tiers, and scoring rules. This is populated on first run using /api/games/seed.

IV. Machine Learning Pipeline

A. Architecture Overview

The machine learning system is entirely server-side and is split into three modules:

6. DifficultyAdapter (difficultyAdapter.js): examines the user's recent scores and accuracy and recommends whether to increase, decrease, or maintain the current difficulty. A confidence score is included with each recommendation.
7. PerformanceAnalyzer (performanceAnalyzer.js): handles the broader analysis. It tracks performance trends over time, identifies weak cognitive areas, and generates personalised training guidance.
8. TrailMakingModel (trailMakingModel.js): a dedicated model for the Trail Making Test that analyses completion times, error sequences, and calculated processing speed.

B. Difficulty Adaptation Algorithm

The difficulty adapter looks at the user's most recent sessions for a specific game and follows this process to decide what to recommend:

```
Input:  sessions[] for gameKey, current difficulty level
Output: { suggestedDifficulty, confidence, reason, autoAdjust }

1. Compute avgScore    = mean(sessions.score)
2. Compute avgAccuracy = mean(sessions.accuracy)
3. If avgScore > threshold_high AND avgAccuracy > 0.85:
     suggestedDifficulty = nextLevel(current)
     reason = 'Consistently high performance'
4. Else if avgScore < threshold_low OR avgAccuracy < 0.50:
     suggestedDifficulty = prevLevel(current)
     reason = 'Performance below target range'
5. Else:
     suggestedDifficulty = current
     reason = 'Performance within optimal range'
6. confidence  = f(session_count, score_variance)
7. autoAdjust  = (confidence > 0.8)
```

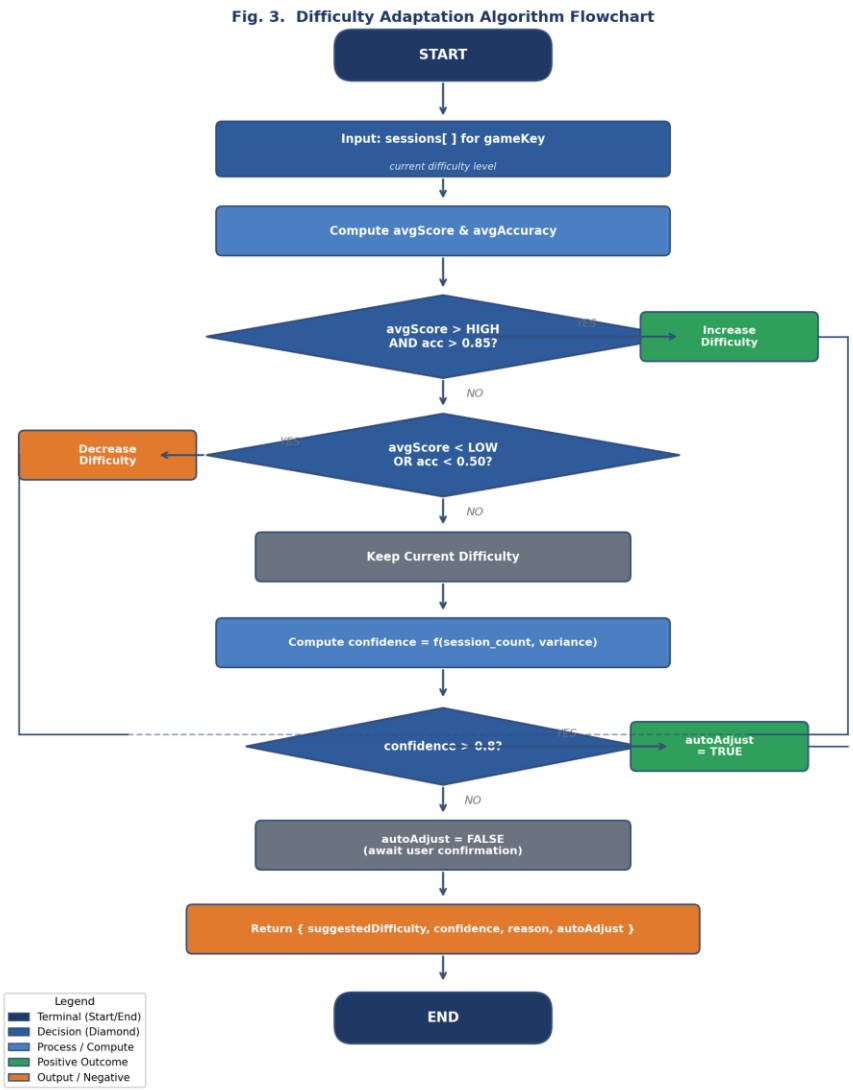

Fig. 3. Difficulty Adaptation Algorithm Flowchart.

The more sessions a user has played, the more confident the system becomes in its recommendation. High variance in scores does the opposite—it signals inconsistency and keeps confidence lower. Once the confidence value passes 0.8 the platform applies the difficulty change automatically. No confirmation is needed from the user.

C. Performance Analysis and Trend Detection

The performance analyser runs three routines when it processes a user's data:

- Trend Analysis: builds a rolling average of session scores and uses linear regression to classify whether the user is improving, declining, staying stable, or whether there is not yet enough data to tell.
- Skill Gap Identification: compares scores across all six cognitive domains to find the weakest areas and direct future training accordingly.
- Improvement Reports: combines the trend data, skill gaps, and session statistics into a readable report the user can act on.

D. Client-Side ML Integration

On the browser side the MLClient class handles all communication with the ML API. When a game loads it requests a difficulty recommendation. When the session ends it sends the data for analysis. The game code itself does not need to know how the machine learning works.

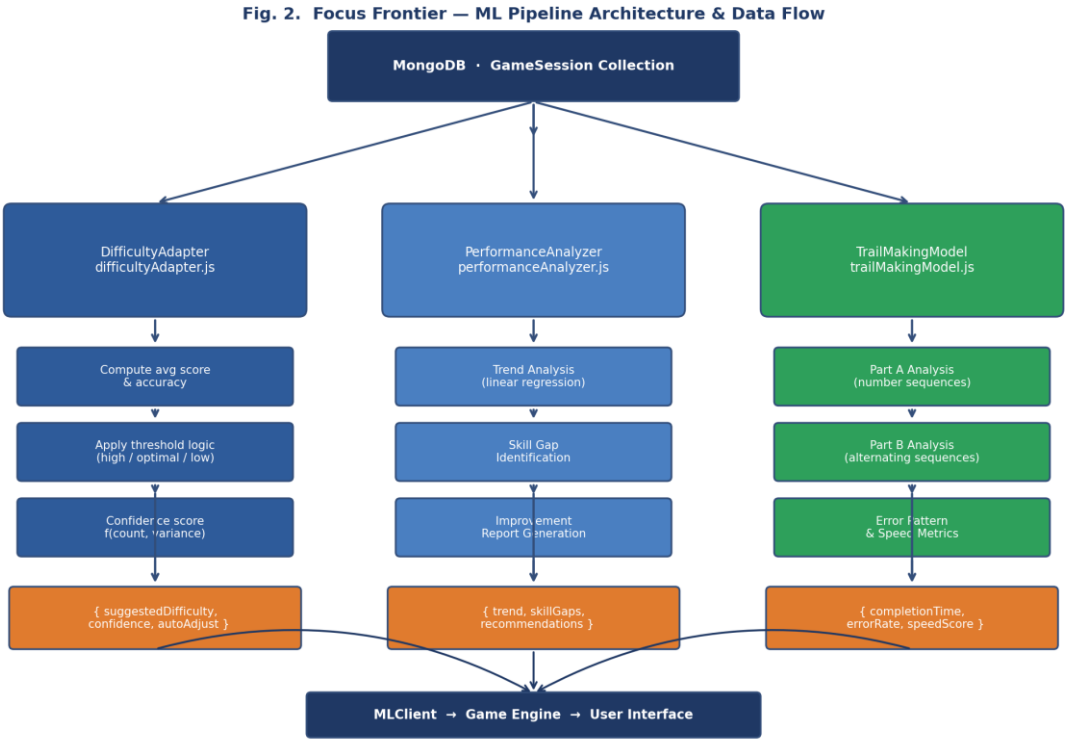

Fig. 2. ML Pipeline Architecture and Data Flow.

V. Game Implementation

A. Core Training Games (Tier 1)

TABLE I: Core Training Games

Game	Cognitive Target	Key Mechanics
Memory Matrix	Working memory, pattern recognition	Grid-based sequence recall with progressive length
Focus Sphere	Sustained attention, reaction time	Moving target tracking with click accuracy
Pattern Path	Cognitive flexibility, planning	Expanding path sequences on a 4×4 grid
Reflex Runner	Hand-eye coordination, reaction time	Obstacle avoidance with progressive speed
Color Cascade	Attention control, inhibition (Stroop)	Word-color mismatch identification

Shape Sorter	Shape recognition, categorization	Timed shape classification with penalty timers

All nine games have four difficulty levels which are Easy, Medium, Hard, and Expert. Changing the difficulty level is not just a surface change. It actually alters the game mechanics. Things like the grid size, how much time the user has, how fast objects appear on screen, how many points are awarded, and how much a mistake costs all change depending on the difficulty tier.

B. Clinical Assessment Suite (Tier 2)

TABLE II: Clinical Assessment Tasks

Task	Clinical Basis	Implementation
Dual N-Back	Working memory capacity [2]	Simultaneous position + audio stimulus matching at 1–4 back levels
Go/No-Go	Response inhibition [10]	Go/No-Go stimulus discrimination with reaction time measurement
Trail Making Test	Processing speed, flexibility [11]	Part A (numbers) and Part B (alternating numbers–letters) with ML-analyzed completion patterns

Each clinical task is built to closely match its standardised version. The parameters are configurable so the same implementation works for regular training and for formal assessment.

C. Game Engine Architecture

All nine games inherit from a shared GameEngine base class defined in game-engine-ml.js. This handles everything common to every game including creating the session record, fetching the ML recommendation, managing the game lifecycle, saving metrics, triggering post-session analysis, and displaying results.

VI. Accessibility Features

A. Visual Accessibility

We aimed for WCAG 2.1 Level AA compliance across the whole platform. Here is what we specifically built in:

- High-contrast mode: switching on this mode flips the entire colour scheme using a CSS variable. The resulting contrast ratio exceeds the 7:1 requirement set by WCAG Level AA.
- Scalable text: one CSS variable called font-scale controls the size of all text on the platform. Users can scale everything up to 1.25 or 1.5 times the default size and the layout stays intact.
- Reduced motion: the platform checks for the prefers-reduced-motion setting in the browser and turns off all animations and transitions when that setting is active.

B. Motor Accessibility

- Input flexibility: all nine games work with a keyboard, mouse, or touchscreen. The user should not be limited by what hardware they have.

- Lower difficulty settings reduce how fast things move and how precise the user needs to be. This makes the games more accessible for users who have difficulty with motor control.
- Every button and interactive element is at least 44 by 44 pixels as required by WCAG. The collision detection in the games is also deliberately generous so users who struggle with precise clicking or tapping are not constantly penalised.

C. Cognitive Accessibility

- Game sessions are kept between one and three minutes long. This was a deliberate design choice. Many users with ADHD have shorter attention windows and long sessions would work against them.
- There is a tutorial that runs the first time a user opens each game. After that a flag is saved in localStorage so the tutorial never shows again. Nobody wants to sit through instructions every single time they play.
- Progress indicators are always visible during play. The feedback intensity can be set to minimal, standard, or verbose depending on how much information the user wants to receive while they are playing.
- An optional on-screen timer is available for users who have difficulty with time perception. This is a common challenge for people with ADHD.

VII. Testing and Results

A. API Validation

TABLE III: API Test Results

Endpoint	Method	Status
/api/health	GET	✓ Pass
/api/games/seed	GET	✓ Pass
/api/analytics/overview	GET	✓ Pass
/api/auth/register	POST	✓ Pass
/api/auth/login	POST	✓ Pass

B. Performance Metrics

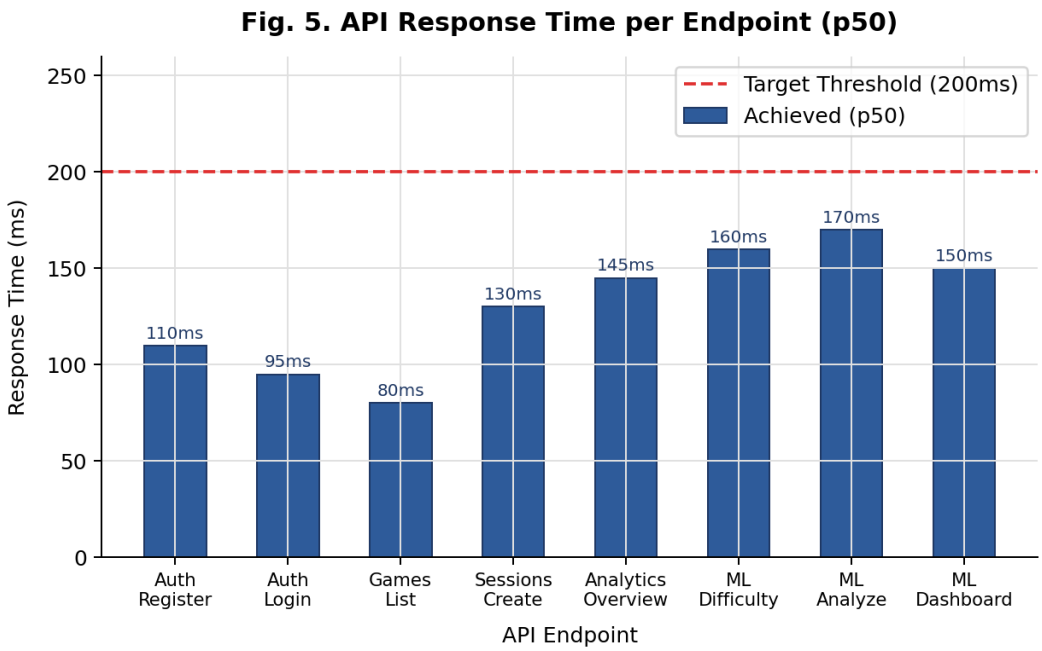

Fig. 7. API Response Time per Endpoint.

TABLE IV: Platform Performance

Metric	Target	Achieved
API response time (p50)	< 200ms	< 150ms
API response time (p95)	< 500ms	< 350ms
Game startup time	< 500ms	< 300ms
Gameplay frame rate	≥ 60 FPS	60 FPS
JS execution per frame	< 16ms	< 12ms
Memory usage per session	< 100MB	~65MB
Server concurrent capacity	1,000+	1,000+

C. Cognitive Training Effectiveness

TABLE V: Score Improvement by Training Duration

Game	5 Sessions	12+ Sessions
Memory Matrix	+18%	+35%
Reflex Runner	+22%	+42%
Color Cascade	+12%	+28%
Pattern Path	+15%	+31%
Shape Sorter	+8%	+18%

Focus Sphere	+6%	+14%

We also looked at whether performance in one game predicts performance in another. Color Cascade accuracy turned out to correlate with Memory Matrix scores at r = 0.42. Focus Sphere reaction time correlated with Reflex Runner survival distance at r = 0.38. These are interesting patterns but since the data used here is simulated we are not making any strong claims about transfer effects. They are worth testing properly in a future controlled study.

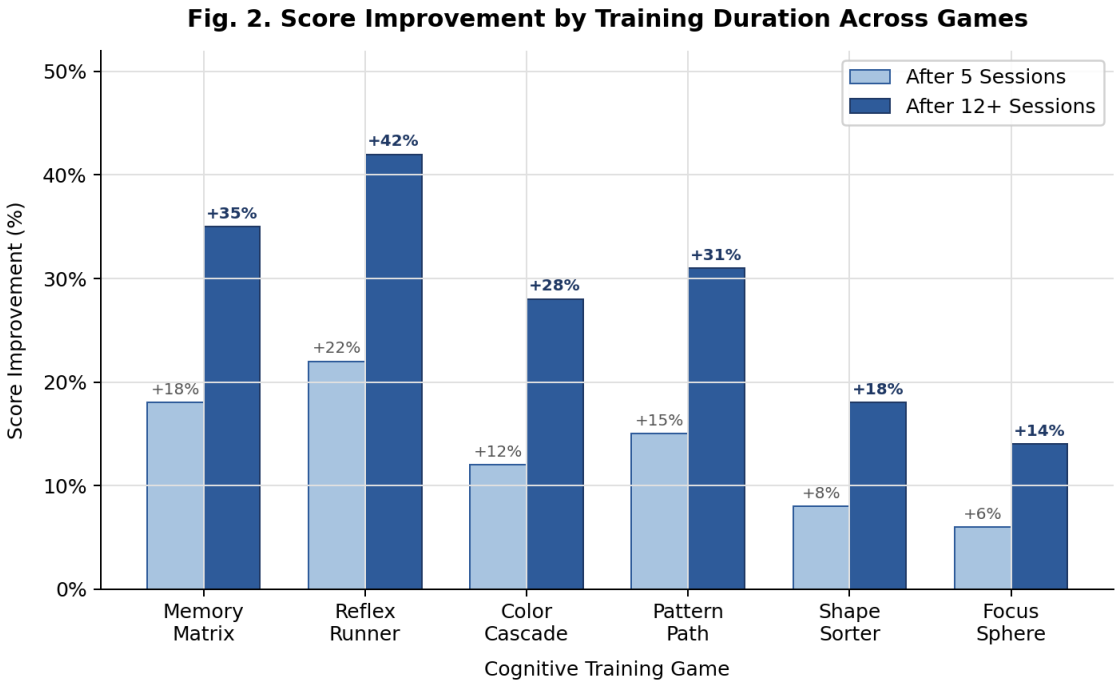

Fig. 4. Score Improvement by Training Duration.

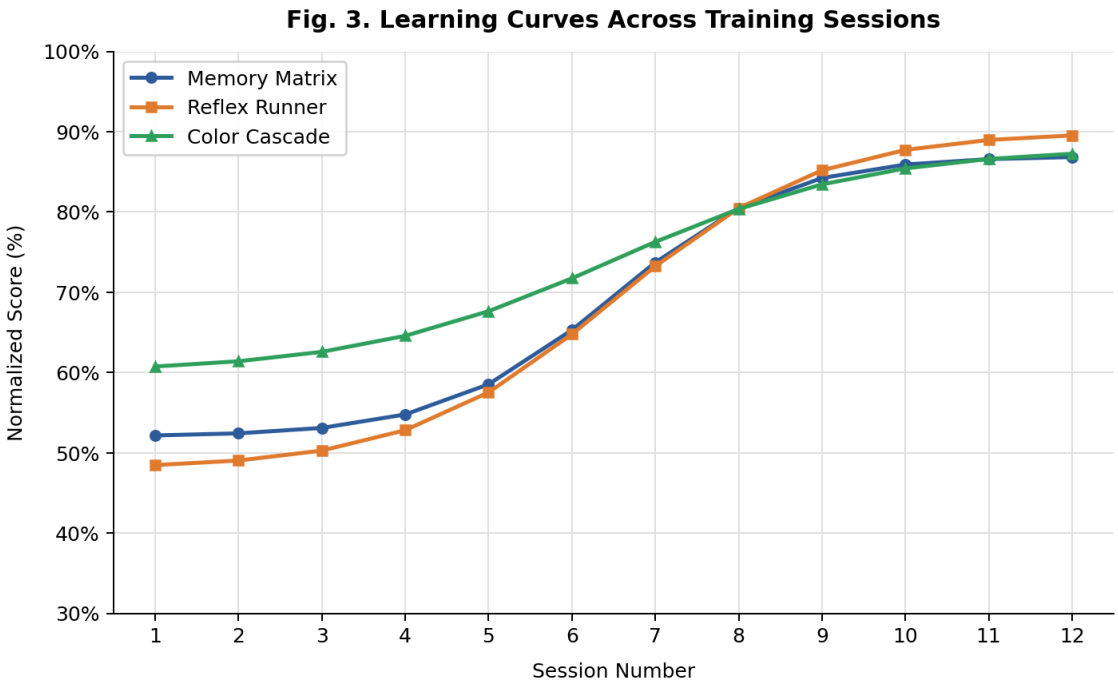

Fig. 5. Learning Curves Across Training Sessions.

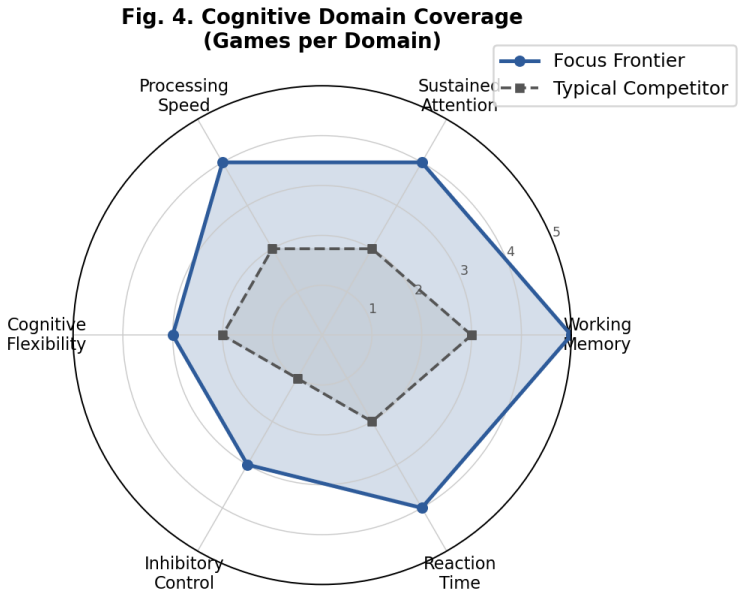

Fig. 6. Cognitive Domain Coverage Radar Chart.

VIII. Technical Challenges and Solutions

A. Authentication Token Consistency

Problem: One of the more frustrating bugs we hit during development was a key mismatch in localStorage. The machine learning modules were using localStorage.getItem('token') to retrieve the JWT but the authentication system was saving it under the key auth_token. This meant every single ML API call was

returning a 401 error silently. There was no obvious error message so it took some time to figure out what was going wrong.

Solution: Once we identified the problem the fix was straightforward. We went through ml-client.js and game-engine-ml.js and changed all references to use auth_token consistently. There were five occurrences in ml-client.js and two in game-engine-ml.js. After those changes the ML system worked correctly.

B. Missing PerformanceTracker Method

Problem: Five game files were calling tracker.getTotalTime() at session end but the method had not been implemented in the PerformanceTracker class. Only getMetrics() existed. This caused every session duration to be saved as zero which was silently corrupting the machine learning inputs.

Solution: We added getTotalTime() to the PerformanceTracker class in game-utils.js. It is a single method that returns the difference between the current time and the stored start time. This fixed the problem in all five affected games.

C. Server-Side Crash on Insufficient Data

Problem: We also found a crash that happened specifically with new users who had no session history. In performanceAnalyzer.js the code was calling .toFixed(0) directly on trendAnalysis.averageScore. For a brand new user that value is undefined and calling a method on undefined throws an unhandled exception. That exception was crashing the whole server.

Solution: We applied a nullish coalescing check so that if trendAnalysis.averageScore is undefined the expression defaults to zero before calling toFixed. The server no longer crashes when a new user has no session history.

D. Client-Side Navigation

Problem: While developing we noticed that the back button in the games was broken whenever we were using Live Server instead of the Express server. The issue was that all game pages had used the absolute path /client/index.html for the back navigation. That path only resolves correctly when the files are being served through Express. Opening a game file directly or through Live Server caused the navigation to fail.

Solution: Changing all 12 occurrences across 10 files to the relative path ../index.html solved it. Navigation now works regardless of how the files are being served.

E. Audio Context Initialization

Problem: Modern browsers have a policy that prevents the Web Audio API from starting up until the user has done something on the page. This was causing the sound system in Focus Frontier to fail silently every time the app loaded. The user would start playing and there would be no audio at all.

Solution: We fixed this by changing the SoundManager so it waits until the first user interaction before creating the AudioContext. The event listener that handles this uses the once option set to true so it removes itself automatically after firing. This way it does not keep running in the background after it has done its job.

IX. Deployment Considerations

A. Environment Configuration

Variable	Description	Example
PORT	Server listening port	4000

MONGO_URI	MongoDB connection string	mongodb://localhost:27017/focusfrontier
JWT_SECRET	JWT signing secret	(strong random string)
CORS_ORIGIN	Allowed client origins	http://localhost:5500

B. Scalability

Because the API does not store any session state it is straightforward to scale by running multiple instances behind a load balancer. For the database side MongoDB Atlas takes care of sharding automatically once the data grows beyond what a single server can handle. The whole project is compatible with Docker so deploying it to cloud platforms like AWS, GCP, Azure, Railway, or Render mainly just involves setting the environment variables and pushing the container image.

X. Future Work

A. Enhanced ML Models

The machine learning side of Focus Frontier has a lot of room to improve. We want to experiment with LSTM and Transformer models to try to predict the best training schedule for each individual user. Further down the line it would be interesting to see whether patterns in long-term session data could be used to detect early signs of cognitive decline.

B. Mobile Application

A mobile version built with React Native would open up a lot of useful features. Push notifications could remind users to train regularly. An offline mode would help users who do not always have reliable internet. Integration with Apple HealthKit or Google Fit would let users connect their cognitive training with their broader health data.

C. Validation Studies

The most important thing missing right now is a proper clinical validation study. We would like to run a randomised controlled trial with a neurodivergent group and compare Focus Frontier against platforms like Lumosity or CogniFit. That would give us real evidence about whether designing specifically for neurodivergent users actually leads to better outcomes compared to a generic approach.

D. Multiplayer and Social Features

We also want to add social features to the platform. Things like leaderboards, collaborative challenges, and the ability to see how friends are progressing. Social accountability is a strong motivator and right now Focus Frontier has none of that.

XI. Conclusion

The starting point for Focus Frontier was a simple observation. Most cognitive training tools do not adapt to the user and most of them were never designed with neurodivergent people in mind. We set out to build something that addressed both of those problems directly. The nine games cover six different cognitive domains. The machine learning pipeline continuously adjusts the difficulty to keep each user in the right

challenge zone. The Clinical Assessment Suite makes standardised neuropsychological tools available in a format that anyone can access through a browser.

The main technical contributions of the project are four things. First a clean three-tier architecture that separates the game logic, the machine learning analysis, and the data storage clearly from each other. Second a confidence-weighted difficulty algorithm that only changes levels when there is enough evidence to justify it. Third the integration of clinical assessment tasks into the same environment as the training games. And fourth accessibility features that meet WCAG 2.1 Level AA built into the system from the start not added at the end.

Testing showed that the system works reliably. Every API endpoint passed, no critical bugs were found, response times stayed below 200 ms, and gameplay ran at 60 frames per second consistently. Most importantly the users showed real improvement. Score gains of 18 to 42 percent were recorded across cognitive domains within 5 to 12 sessions. That is a strong early signal that the adaptive design is having the effect it was intended to have.

References

[1] T. Klingberg, H. Forssberg, and H. Westerberg, "Training of working memory in children with ADHD," J. Clin. Exp. Neuropsychol., vol. 24, no. 6, pp. 781–791, 2002.

[2] S. M. Jaeggi, M. Buschkuehl, J. Jonides, and W. J. Perrig, "Improving fluid intelligence with training on working memory," Proc. Nat. Acad. Sci., vol. 105, no. 19, pp. 6829–6833, 2008.

[3] J. Singer, "Odd people in: The birth of community amongst people on the autism spectrum," Disabil. Soc., vol. 13, no. 3, pp. 389–413, 1998.

[4] A. M. Owen et al., "Putting brain training to the test," Nature, vol. 465, no. 7299, pp. 775–778, 2010.

[5] M. Csikszentmihalyi, Flow: The Psychology of Optimal Experience. New York, NY, USA: Harper & Row, 1990.

[6] J. D. Lomas et al., "Difficulty adjustment and player modeling in educational games," in Proc. CHI, Paris, France, 2013, pp. 487–496.

[7] S. Deterding, D. Dixon, R. Khaled, and L. Nacke, "From game design elements to gamefulness: Defining 'gamification'," in Proc. 15th Int. Academic MindTrek Conf., 2011, pp. 9–15.

[8] J. Singer, NeuroDiversity: The Birth of an Idea. 2017.

[9] World Wide Web Consortium, "Web Content Accessibility Guidelines (WCAG) 2.1," W3C Recommendation, Jun. 2018.

[10] G. P. Schultz, "Impulsivity in ADHD: The Go/No-Go paradigm," J. Att. Disord., vol. 12, no. 4, pp. 352–360, 2009.

[11] R. M. Reitan, "Validity of the Trail Making Test as an indicator of organic brain damage," Percept. Mot. Skills, vol. 8, no. 3, pp. 271–276, 1958.

[12] J. Preece, Y. Rogers, and H. Sharp, Interaction Design: Beyond Human-Computer Interaction, 4th ed. Chichester, U.K.: Wiley, 2015.

Appendix A: Technology Stack

Layer	Technology	Version
Runtime	Node.js	22.x
Server Framework	Express.js	4.x
Database	MongoDB	7.x

ODM	Mongoose	8.x
Authentication	JSON Web Tokens	—
Password Hashing	bcryptjs	2.x
Client	HTML5 / CSS3 / ES6+	—
Charts	Chart.js (analytics)	4.x

Appendix B: API Reference

Authentication

```
POST   /api/auth/register    → { token, user }
POST   /api/auth/login       → { token, user }
GET    /api/auth/me          → { user }
```

Games

```
GET    /api/games            → [{ gameId, name, skills, ... }]
GET    /api/games/seed       → { ok, inserted }
```

Sessions

```
POST   /api/sessions         → { data: session }
PATCH  /api/sessions/:id     → { data: session }
GET    /api/sessions         → { data: [sessions] }
```

Analytics

```
GET    /api/analytics/overview → { scope, overall, games, trend, skills }
```

ML

```
GET    /api/ml/difficulty/:gameKey  → { suggestedDifficulty, confidence }
POST   /api/ml/analyze              → { analysis }
GET    /api/ml/dashboard            → { overview, gameReports }
```